

ÉPREUVE ÉCRITE D'ALGORITHMIQUE PARALLÈLE

Telecom Nancy – Université Lorraine, 2024-2025

2 feuilles de notes recto/verso au format A4 autorisées – Aucun appareil électronique

Pour chaque question, il est demandé de bien identifier le numéro d'exercice et de question (ex : 1.c pour la question c de l'exercice 1). Les réponses aux questions doivent être **concises mais argumentées**. Lorsqu'une valeur numérique est attendue, le **détail des calculs** et une valeur approchée doivent être fournis. Tous les *algorithmes* demandés doivent être écrits en **langage algorithmique** avec les éléments de parallélisme vus en cours (boucles parallèles, opérateurs α et β , communications,...). Il est conseillé de lire le sujet entièrement avant de commencer.

△ Dans tout le sujet, on redéfinit les opérateurs α et β avec un paramètre explicite n pour la taille du/des tableau(x) : $\alpha(op, v1, v2, n)$ et $\beta(op, v, n)$

Les opérations arithmétiques, l'ajout d'un élément en fin de liste, le déverrouillage ont un coût 1 (constant). L'opération de verrouillage a un coût maximal égal au nombre d'unités de calcul uniquement lorsque les unités verrouillent en même temps.

Exercice 1 : Recherche d'un motif dans un texte en mémoire partagée

On souhaite construire la liste des positions d'un motif (chaîne de caractères) de longueur m dans un texte txt de longueur n ($n \geq m$).

On propose l'algorithme séquentiel suivant :

```
// Indique si le motif de taille m est présent à la position pos dans le texte txt de taille n
fonction testePosition(pos : entier, motif : chaîne, m : entier, txt : chaîne, n : entier) : booléen
Début
  res ← Vrai // On considère au départ que le motif est à la position pos dans txt
  i ← 0      // On parcourt le motif
  tant que res et i < m et pos + i < n faire // Boucle de parcours
    res ← (motif[i] = txt[pos+i])           // Comparaison des caractères
    i ← i + 1                               // Passage au caractère suivant
  ftant
  retourner res                             // Renvoi du résultat
Fin

// Construit la liste des positions où se trouve le motif de taille m dans le texte txt de taille n
fonction rechercheMotif(motif : chaîne, m : entier, txt : chaîne, n : entier) : liste d'entiers
Début
  liste ← ∅                                 // Liste vide au départ
  pour i de 0 à n-m faire                  // Parcours des positions initiales possibles
    si testePosition(i, motif, m, txt, n) alors // Test si le motif est à la position i
      liste ← liste ∪ {i}                  // Ajout de la position en fin de liste
    fsi
  fpour
  retourner liste                          // Renvoi du résultat
Fin
```

- Donnez le temps séquentiel total de `rechercheMotif` en fonction des tailles de motif et de texte.
- Proposez une version parallèle de `testePosition` avec un maximum de parallélisme sachant que l'on dispose de la fonction `sousTexte(txt : chaîne, pos : entier) : chaîne` qui extrait en temps constant le sous-texte de txt commençant à la position pos .
- Donnez le nombre optimal d'unités à utiliser, les temps séquentiel et parallèle et l'accélération.
- Proposez une version parallèle de `rechercheMotif` utilisant P processus avec répartition cyclique des données et construction d'une liste unique, partagée entre les processus.
- Donnez le temps de votre algorithme en considérant que le test de position du motif est séquentiel avec un coût m .

Exercice 2 : Recherche d'un motif dans un texte en mémoire distribuée

Nous avons vu en TP le modèle distribué maître – travailleurs pour compter les occurrences des lettres dans un texte.

a) Donnez deux inconvénients potentiels d'un tel modèle ?

On considère un texte `txt` de taille `n`, contenu entièrement sur le premier processus (numéro 0) et on souhaite construire la liste des positions (s'il y en a) d'un motif donné, de taille `m`, en utilisant la stratégie suivante :

- **Distribution par blocs maximaux** du texte sur l'ensemble des processus (premier compris)
- **Recherche** du motif dans le texte local pour chaque processus
- **Fusion** des résultats (listes de positions) sur le processus 0

b) Proposez un algorithme qui distribue le texte sur les processus depuis le processus 0 sachant que les autres processus ne connaissent pas la taille du texte initial.

➡ On considère que les variables `numP` et `nbP` sont déjà définies et contiennent respectivement le numéro du processus courant et le nombre total de processus.

➡ Il peut être plus simple d'envoyer la taille des données à envoyer avant les données elles-mêmes.

➡ On dispose de la fonction `tailleDe(...)` pour récupérer la taille d'un type de données, de la fonction `sousTexte(...)` définie dans l'exercice 1 pour récupérer le texte à une position donnée et de la fonction `allouer(taille : entier) : tableau` qui alloue un tableau de taille donnée.

c) Proposez un algorithme pour fusionner sur le processus 0, les listes construites localement sur chaque processus.

△ Une liste n'est pas une suite d'éléments nécessairement contigus en mémoire et ne peut donc pas être envoyée en une seule fois. Il faut soit envoyer les éléments un à un, soit la recopier dans un tableau que l'on peut envoyer en une fois.